

Computer Organization and Assembly Language Chapter 6 Conditional Processing

Branches

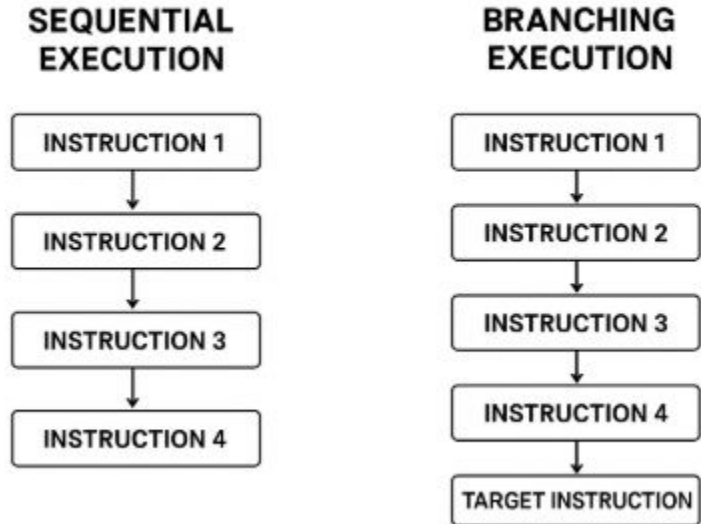
In **assembly language**, **branches** are instructions that change the normal flow of execution in a program.

Normally, the CPU executes instructions one after another (sequential flow). But with **branch instructions**, the CPU can "jump" to a different instruction address, depending on conditions or unconditionally.

A **branch** in assembly is any instruction that alters the no conditionally (based on flags) or unconditionally (always).

Types of Branches:

- Unconditional Branch (Jump)
- Conditional Branch Jump based on Flags/Condition
- Branch with Procedure Calls / Returns



Conditional Branching

- A programming language that permits decision making lets you alter the flow of control, using a technique known as **Conditional Branching**
- Every **IF** statement, **switch** statement, or **conditional loop** found in high-level languages has built-in branching logic.
- Assembly language as it is, provides all the tools you need for decision-making logic.

Boolean and Comparison Instructions

- Using the conditional instructions to conditional loops and if-then–else structures requires an understanding of the flags registers.
- The flags register is affected by most instruction as a byproduct of the operation.
 - There are some instruction whose whole purpose is to change the flags register.
 - These include **CMP**, **AND**, **OR**, **XOR**, **NOT**, and **NEG**.

Status Flags - Review

- The **Zero** flag is set when the result of an operation equals zero.
- The **Carry** flag is set when an instruction generates a result that is too large (or too small) for the destination operand.
- The **Sign** flag is set if the destination operand is negative, and it is clear if the destination operand is positive.
- The **Overflow** flag is set when an instruction generates an invalid signed result (bit 7 carry is XORed with bit 6 Carry).
- The **Parity** flag is set when an instruction generates an even number of 1 bits in the low byte of the destination operand.
- The **Auxiliary Carry** flag is set when an operation produces a carry out from bit 3 to bit 4

NOT Instruction

- The NOT instruction toggles (inverts) all bits in an operand. The result is called the *one's complement*.
- The following operand types are permitted:

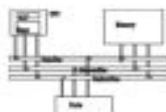
NOT reg

NOT mem

- For example, the one's complement of F0h is 0Fh:

mov al,11110000b

not al ; AL = 00001111b



NOT instruction

- Performs a bitwise Boolean NOT operation on a single destination operand
- Syntax: (no flag affected)

`NOT destination`

- Example:

```
mov al, 11110000b
not al
```

NOT 0 0 1 1 1 0 1 1
 1 1 0 0 0 1 0 0 ——— inverted

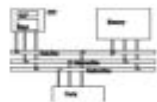
NOT

X	$\neg X$
F	T
T	F

AND Instruction

- The **AND** instruction performs a **boolean (bitwise) AND operation** between each pair of matching bits in two operands and places the result in the destination operand:
AND destination,source
- The following operand combinations are permitted:
AND reg,reg
AND reg,mem
AND reg,imm
AND mem,reg
AND mem,imm
- The operands can be 8, 16, or 32 bits, and they must be the same size.
- For each matching bit in the two operands, the following rule applies:
 - If both bits equal 1, the result bit is 1; otherwise, it is 0

AND instruction

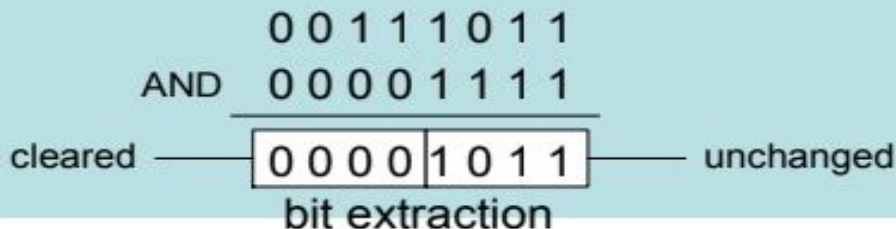


- Performs a bitwise Boolean AND operation between each pair of matching bits in two operands
- Syntax: (O=0,C=0,SZP)

AND destination, source

- Example:

```
mov al, 00111011b
and al, 00001111b
```



AND

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

Example AND

- For example, suppose AL is initially set to 10101110 binary.
- After ANDing it with 11110110, AL equals 10100110:

```
mov al,10101110b
```

```
and al,11110110b ; result in AL = 10100110
```

OR Instruction

- The OR instruction performs a **boolean OR operation** between each pair of matching bits in two operands and places the result in the destination operand:

OR destination,source

- The OR instruction uses the same operand combinations as the AND instruction:

OR reg,reg

OR reg,mem

OR reg,imm

OR mem,reg

OR mem,imm

- The operands can be 8, 16, or 32 bits, and they must be the same size. For each matching bit in the two operands, the output bit is 1 when at least one of the input bits is 1.



OR instruction

- Performs a bitwise Boolean OR operation between each pair of matching bits in two operands
- Syntax: (O=0,C=0,SZP)

OR destination, source

- Example:

```
mov dl, 00111011b
```

```
or dl, 00001111b
```

unchanged —

0	0	1	1
1	1	1	1

 — set

OR

x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

XOR instruction



- Performs a bitwise Boolean exclusive-OR operation between each pair of matching bits in two operands
- Syntax: (O=0,C=0,SZP)

XOR destination, source

- Example:

```
mov dl, 00111011b
```

```
xor dl, 00001111b
```

0 0 1 1 1 0 1 1
XOR 0 0 0 0 1 1 1 1
—————
unchanged ——— 0 0 1 1 | 0 1 0 0 ——— inverted

XOR

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

XOR is a useful way to invert the bits in an operand and data encryption

Applications (1 of 4)

- Task: Convert the character in AL to upper case.
- Solution: Use the AND instruction to clear bit 5.

<code>mov al, 'a'</code>	<code>; AL = 01100001b</code>
<code>and al, 11011111b</code>	<code>; AL = 01000001b</code>

- We convert lower case to upper case by clearing bit 5:

0	1	1	0	0	0	0	1	{ 'a' }
0	1	0	0	0	0	0	1	{ 'A' }

```
.data
array BYTE 50 DUP(?)
.code
    mov     ecx, LENGTHOF array
    mov     esi, OFFSET array
L1:
    and     byte ptr [esi], 11011111b
    inc     esi
    loop    L1
```

Applications (2 of 4)



- Task: Convert a binary decimal byte into its equivalent ASCII decimal digit.
- Solution: Use the OR instruction to set bits 4 and 5.

```
mov al,6          ; AL = 00000110b
or  al,00110000b  ; AL = 00110110b
```

The ASCII digit '6' = 00110110b

- OR can be used to convert a one-digit value into its ASCII equivalent:

```
mov dl, 5          ; binary value
or  dl, 30h        ; convert to ASCII
```

- ORing a value with itself preserves the value but sets to flags

```
- ZF = 1          if AL = 0
- SF = 1          if AL < 0
- SF = ZF = 0     if AL > 0
or al, al         ; sets the flags
```

Applications (3 of 4)



- Task: Jump to a label if an integer is even.
- Solution: AND the lowest bit with a 1. If the result is Zero, the number was even.

```
mov ax,wordVal
and ax,1          ; low bit set?
jz  EvenValue     ; jump if Zero flag set
```

We use the **AND** operation with **1** (binary **0001**):

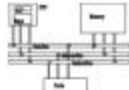
- **number AND 1** extracts only the LSB.
- If result = **0** → number is **even**.
- If result = **1** → number is **odd**.

(15)1111 AND 0001
= 0001

Examples:

- **6 (0110) AND 0001 → 0000 → even**
- **7 (0111) AND 0001 → 0001 → odd**

Applications (4 of 4)



- Task: Jump to a label if the value in AL is not zero.
- Solution: OR the byte with itself, then use the JNZ (jump if not zero) instruction.

```
or    al,al  
jnz   IsNotZero    ; jump if not zero
```



TEST instruction

- Performs a nondestructive **AND** operation between each pair of matching bits in two operands
- No operands are modified, but the flags are affected.
- Example: jump to a label if either bit 0 or bit 1 in AL is set.

```
test al,00000011b  
jnz  ValueFound
```

- Example: jump to a label if neither bit 0 nor bit 1 in AL is set.

```
test al,00000011b  
jz   ValueNotFound
```

TEST Instruction

- The **TEST** instruction performs an implied AND operation between corresponding bits in the two operands and sets the flags without modifying either operand.
- The format for the **TEST** instruction is:

TEST *reg, reg*

TEST *reg, mem*

TEST *reg, imm*

TEST *mem, reg*

TEST *mem, imm*

reg, mem, and imm can be 8, 16, or 32 bits.

CMP instruction (1 of 3)



- Compares the destination operand to the source operand
 - Nondestructive subtraction of source from destination (destination operand is not changed)
- Syntax: (OSZCAP)
CMP destination, source
- Example: destination == source

```
mov al,5  
cmp al,5           ; Zero flag set
```

- Example: destination < source

```
mov al,4  
cmp al,5           ; Carry flag set
```

CMP effect on these flags:

- **OF (Overflow Flag)**: Set if signed overflow occurs in subtraction.
- **SF (Sign Flag)**: Set if the result is negative (MSB = 1).
- **ZF (Zero Flag)**: Set if both operands are equal (result = 0).
- **CF (Carry Flag)**: Set if an unsigned borrow occurs (i.e., $\text{src} > \text{dest}$ in unsigned terms).
- **AF (Auxiliary Carry Flag)**: Set if there's a borrow out of bit 3 (used in BCD operations).
- **PF (Parity Flag)**: Set if the result has even parity (even number of 1 bits in the low byte).

👉 So the **syntax (OSZCAP)** means that the **CMP** instruction **updates all these flags** accordingly.

● Case 1: AL = 5

asm

```
MOV AL, 5
```

```
CMP AL, 5 ; internally 5 - 5 = 0
```

- Result (internal): 0
- Flags:
 - ZF = 1 (equal → zero result)
 - SF = 0 (not negative)
 - CF = 0 (no borrow, $5 \geq 5$)
 - OF = 0 (no signed overflow)
- AL remains 5

● Case 2: AL = 3

asm

```
MOV AL, 3
```

```
CMP AL, 5      ; internally 3 - 5 = -2 (0xFE in 8-bit)
```

- Result (internal): -2 → 11111110b
- Flags:
 - ZF = 0 (not equal)
 - SF = 1 (result negative)
 - CF = 1 (borrow occurred since 5 > 3 in unsigned)
 - OF = 0 (no signed overflow, within range -128..127)
- AL remains 3

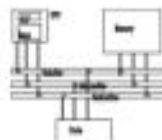
Case 3: AL = 8

asm

```
MOV AL, 8
```

```
CMP AL, 5      ; internally 8 - 5 = 3
```

- Result (internal): 3 → 00000011b
- Flags:
 - ZF = 0 (not equal)
 - SF = 0 (positive)
 - CF = 0 (no borrow, 8 > 5 in unsigned)
 - OF = 0 (no signed overflow)
- AL remains 8



CMP instruction (2 of 3)

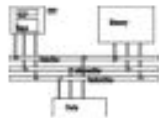
- Example: destination > source

```
mov al,6  
cmp al,5           ; ZF = 0, CF = 0
```

(both the Zero and Carry flags are clear)

The comparisons shown so far were unsigned.

CMP instruction (3 of 3)



The comparisons shown here are performed with signed integers.

- Example: destination > source

```
mov al,5  
cmp al,-2          ; Sign flag == Overflow flag
```

- Example: destination < source

```
mov al,-1  
cmp al,5           ; Sign flag != Overflow flag
```

Conditions

unsigned	ZF	CF
destination<source	0	1
destination>source	0	0
destination=source	1	0

signed	flags
destination<source	SF != OF
destination>source	SF == OF
destination=source	ZF=1

Setting and Clearing the Zero Flag

and al, 0 ; Set Zero Flag

or al, 1 ; Clear Zero Flag

- Any value ANDed with 0 = 0.
- So AL becomes 0.

Flags updated:

- ZF = 1 → because result is 0.
- SF = 0 → sign bit of 0 = 0.
- PF = 1 → parity of 0 (no 1s, so even parity).
- OF = 0, CF = 0 → AND clears them.



This is a common trick to **set the Zero Flag** without using **CMP**.

Setting and Clearing the Zero Flag

and al, 0 ; Set Zero Flag

or al, 1 ; Clear Zero Flag

- AL is currently 0.
- `0 OR 1 = 1`, so AL becomes 1.

Flags updated:

- `ZF = 0` → result not zero.
- `SF = 0` → result positive.
- `PF = 0` → `00000001b` has one "1" (odd parity).
- `OF = 0, CF = 0` → `OR` clears them.

👉 This is a trick to clear the Zero Flag.

Setting and Clearing the Sign Flag

or al, 80h ; Set Sign Flag

and al, 7fh ; Clear Sign Flag

- `80h = 10000000b` (only MSB = 1).
- OR'ing any value with `80h` will **force the MSB (sign bit) = 1**.
- That means the result will be interpreted as **negative** in signed arithmetic.

Flags:

- **SF = 1** → because sign bit is set.
- **ZF = 0** (unless AL was 0x80 before and stays 0x80, then still ≠ 0).
- **PF** updated based on result's low byte.
- **OF = 0, CF = 0** → logical instructions clear them.

👉 Trick: **Set the Sign Flag** by forcing MSB = 1.

Setting and Clearing the Sign Flag

or al, 80h ; Set Sign Flag

and al, 7fh ; Clear Sign Flag

- **7Fh = 01111111b** (all bits except MSB = 1).
- AND'ing any value with **7Fh** will **force the MSB (sign bit) = 0**.
- That makes the result **non-negative** in signed arithmetic.

Flags:

- **SF = 0** → sign bit cleared.
- **ZF = 1** if result = 0 (e.g., if AL was 80h before masking).
- **OF = 0, CF = 0** → cleared by AND.
- **PF** updated.



Trick: Clear the Sign Flag by forcing MSB = 0.

- Setting and Clearing the Carry Flag

`stc` ; Set Carry Flag

`clc` ; Clear Carry Flag

- Setting and Clearing the Overflow Flag

`mov al, 7fH` ; AL = +127

`inc al` ; AL = 80H; OF = 1

`or eax, 0` ; Clear Overflow
; Flag

```
mov    al, 7fH ; AL = +127
inc    al      ; AL = 80H; OF = 1
```

- $01111111b + 1 = 10000000b$
- $AL = 80h = -128$ (signed).

Now check the **flags**:

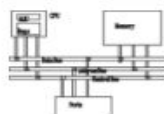
- **OF = 1** → because result went from +127 to -128, which is a signed overflow.
- **SF = 1** → sign bit = 1 (result negative).
- **ZF = 0** → result not zero.
- **CF = unchanged** (INC does not affect Carry).
- **PF, AF** updated according to result (not the focus here).

```
OR EAX, 0 ; logical OR with 0
```

- This instruction **does not change the value of EAX**.
- But it updates **all status flags** except CF and OF.
 - Since result = AL = `80h`, EAX unchanged.
 - OF = **cleared (set to 0)** because `OR` always clears Overflow.
 - CF = **cleared (set to 0)** by `OR`.
 - SF = 1 (still negative).
 - ZF = 0 (still nonzero).
 - PF updated based on parity of low byte.

So effectively, `OR reg, 0` is a common trick to **clear OF and CF** while preserving the actual register contents.

Conditional structures



- There are no high-level logic structures such as if-then-else, in the IA-32 instruction set. But, you can use combinations of comparisons and jumps to implement any logic structure.
- First, an operation such as **CMP**, **AND** or **SUB** is executed to modified the CPU flags. Second, a conditional jump instruction tests the flags and changes the execution flow accordingly.

```
        CMP AL, 0
        JZ  L1
        :
L1:
```

$\mathcal{J}_{cond}$ Instruction

- A conditional jump instruction branches to a destination label when a flag condition is true.
- If the flag is false, the instruction immediately following the conditional jump is performed instead.
- The syntax is:
 $\mathcal{J}_{cond} \text{ destination}$

- Compare AL to Zero. Jump to L1 if the zero flag was set by the comparison:

```
    cmp  al, 0
    jz   L1
    ... ..
```

L1:

- Perform a bitwise AND on the DL register .
Jump to L2 if the Zero flag is clear:

```
    and  dl, 10110000b
    jnz  L2
    ... ..
```

L2:

Conditional Jumps

- *Using the CMP Instruction Suppose you want to jump to label L1 when EAX equals 5.*
- *In the next example, if EAX equals 5, the CMP instruction sets the Zero flag; then, the JE instruction jumps to L1 because the Zero flag is set:*

`cmp eax,5`

`je L1 ; jump if equal`

Conditional Jumps

- In the following example, the JL instruction jumps to label L1 because AX is less than 6:

```
mov ax,5
```

```
cmp ax,6
```

```
jl L1 ; jump if less
```

- In the following example, the jump is taken because AX is greater than 4:

```
mov ax,5
```

```
cmp ax,4
```

```
jg L1 ; jump if greater
```

Types of Conditional Jump Instructions

- The x86 instruction set has a large number of conditional jump instructions.
- They are able to compare signed and unsigned integers and perform actions based on the values of individual CPU flags.
- **The conditional jump instructions can be divided into four groups:**
 - Jumps based on specific flag values
 - Jumps based on equality between operands or the value of (E)CX
 - Jumps based on comparisons of unsigned operands
 - Jumps based on comparisons of signed operands

Jumps based on specific flags



Mnemonic	Description	Flags
JZ	Jump if zero	ZF = 1
JNZ	Jump if not zero	ZF = 0
JC	Jump if carry	CF = 1
JNC	Jump if not carry	CF = 0
JO	Jump if overflow	OF = 1
JNO	Jump if not overflow	OF = 0
JS	Jump if signed	SF = 1
JNS	Jump if not signed	SF = 0
JP	Jump if parity (even)	PF = 1
JNP	Jump if not parity (odd)	PF = 0

Table 6-3 Jumps Based on Equality.

Mnemonic	Description
JE	Jump if equal (<i>leftOp = rightOp</i>)
JNE	Jump if not equal (<i>leftOp $\neq$ rightOp</i>)
JCXZ	Jump if CX = 0
JECXZ	Jump if ECX = 0

```

mov  al,+127           ; hexadecimal value is 7Fh
cmp  al,-128           ; hexadecimal value is 80h
ja   IsAbove           ; jump not taken, because 7Fh < 80h
jg   IsGreater         ; jump taken, because +127 > -128

```

Jumps based on Unsigned Comparisons

Mnemonic	Description	Flag(s)
JA	Jump if above ($op1 > op2$)	$CF = 0 \ \& \ ZF = 0$
JNBE	Jump if not below or equal	$CF = 0 \ \& \ ZF = 0$
JAE	Jump if above or equal	$CF = 0$
JNB	Jump if not below	$CF = 0$

Jumps based on Unsigned Comparisons

Mnemonic	Description	Flag(s)
JB	Jump if below ($op1 < op2$)	CF = 1
JNAE	Jump if not above	CF = 1
JBE	Jump if below or equal	CF = 1 or ZF = 1
JNA	Jump if not above	CF = 1 or ZF = 1

Jumps based on Signed Comparisons

Mnemonic	Description	Flag(s)
JG	Jump if greater	SF = 0 & ZF = 0
JNLE	Jump if not less than or equal	SF = 0 & ZF = 0
JGE	Jump if greater than or equal	SF = OF
JNL	Jump if not less than	SF = OF

Jumps based on Signed Comparisons

Mnemonic	Description	Flag(s)
JL	Jump if less	SF < > OF
JNGE	Jump if not greater than or equal	SF < > OF
JLE	Jump if less than or equal	ZF = 1 or SF < > OF
JNG	Jump if not greater than	ZF = 1 or SF < > OF

Jumps based on Signed Comparisons

Mnemonic	Description	Flag(s)
JS	Jump if signed (op1 is negative)	SF = 1
JNS	Jump if not signed	SF = 0
JO	Jump if overflow	OF = 1
JNO	Jump if not overflow	OF = 0

Conditional Jumps Applications

- Testing Status Bits

Suppose, for example, that an 8-bit memory operand named *STATUS* contains status information about an external device attached to the computer. The following instructions jump to a label if bit 5 is set, indicating that the device is offline:

Testing a single bit

```
mov al, status
test al, 00100000b    ; logical AND: AL & 20h
jnz EquipOffline      ; if result ≠ 0 → bit 5 = 1
```

- **TEST** works like **AND**, but **does not change AL** (nondestructive).
- Only flags are updated.
- If bit 5 = 1, then result ≠ 0 → ZF = 0 → JNZ taken → jump to EquipOffline.
- If bit 5 = 0, ZF = 1 → no jump.

👉 Application: Check if equipment is **offline** (status bit = 1).

Testing multiple bits

```
mov al, status
test al, 00010011b    ; check bits 0,1,4
jnz InputDataByte     ; if any of these bits = 1
```

- Here, mask = `00010011b`.
- If any of bits 0, 1, or 4 are set $\rightarrow$ result $\neq 0 \rightarrow$ **JNZ taken**.
- If none of them are set $\rightarrow ZF = 1 \rightarrow$ no jump.

👉 Application: If there's **input data available** (any relevant flag set), branch to handler.

Checking if all selected bits are set

```
mov al, status
and al, 10001100b    ; preserve only bits 2,3,7
cmp al, 10001100b    ; are all these bits set?
je ResetMachine      ; if yes, jump
```

Step by step:

1. **AND** clears all bits except 2,3,7.
2. If `AL == 10001100b` after masking → it means bits 2,3,7 are **all ON**.
3. **CMP** sets **ZF = 1** if equal.
4. **JE** (Jump if Equal) branches if all bits match.

👉 Application: If machine reports **all required conditions (bits set)**, then branch to reset routine.

Example – Larger of Two Integers

```
mov dx, ax      ; assume AX is larger
cmp ax, bx      ; compare AX with BX
jae L1          ; if AX >= BX, jump to L1
mov dx, bx      ; else, BX is larger → put BX in DX
L1:             ; here DX contains the larger integer
```

Example – Smallest of Three Integers

```
.data
V1 WORD ?
V2 WORD ?
V3 WORD ?

.code
mov ax, V1      ; assume V1 is the smallest → AX = V1
cmp ax, V2      ; compare AX with V2
jbe L1          ; if AX ≤ V2, keep AX (smallest so far)
mov ax, V2      ; else, V2 < AX → AX = V2
L1: cmp ax, V3   ; compare AX with V3
jbe L2          ; if AX ≤ V3, keep AX
mov ax, V3      ; else, V3 < AX → AX = V3
L2:             ; here AX contains the smallest value
```

Find the first even number in an array of unsigned integers

```
.date
intArray DWORD 7,9,3,4,6,1
.code
...
    mov     ebx, OFFSET intArray
    mov     ecx, LENGTHOF intArray
L1:   test   DWORD PTR [ebx], 1
      jz     found
      add    ebx, 4
      loop   L1
...
```

- EBX = address of first element of the array.
- ECX = 6 (the number of elements).
- Will be used with the `LOOP` instruction.
- `test DWORD PTR [ebx], 1`
 - Performs bitwise AND with `1`.
 - Effect: checks the **least significant bit (LSB)**.
 - If $LSB = 1 \rightarrow$ number is **odd**.
- If not even, move EBX to the next element.
- Since each element is a DWORD (4 bytes), add 4.
- Decrement ECX.
- If $ECX \neq 0$, repeat loop.
- If $ECX = 0$, stop (end of array reached).

Example – Scanning An Array

```
TITLE Scanning an Array (ArrayScan.asm)
; Scan an array for the first nonzero value.
INCLUDE      Irvine32.inc

.data
intArray     SWORD 0, 0, 1, 20, 35, -12, 66, 4, 0
noneMsg      BYTE  "A nonzero value wasnt found", 0
.code
main PROC
    mov     ebx, OFFSET intArray
                                ; point to the array
    mov     ecx, LENGTHOF intArray
                                ; loop counter
```

```
L1:    cmp    word ptr [ebx], 0
        jnz    found            ; found a value
        add    ebx, 2           ; point to next
        loop   L1               ; continue the loop
        jmp    notFound        ; none found
```

```
found:
        movsx  eax, word ptr [ebx]
        call   WriteInt
        jmp    quit
```

```
notFound:            ; display "not found message"
        mov    edx, OFFSET noneMsg
        call   WriteString
```

```
quit: call    CrLf
        exit
```

```
main  ENDP
      END    main
```

Simple String Encryption

If an integer X is XORed with Y and the resulting value is XORed with Y again, the value produced is X :

$$((X \otimes Y) \otimes Y) = X$$

This “reversible” property of XOR provides an easy way to perform a simple form of data encryption: A *plain text* message is transformed into an encrypted string called *cipher text* by XORing each of its characters with a character from a third string called a *key*. The intended viewer can use the key to decrypt the cipher text and produce the original plain text.

1. The user enters the plain text.
2. The program uses a single-character key to encrypt the plain text, producing the cipher text, which is displayed on the screen.
3. The program decrypts the cipher text, producing and displaying the original plain text.

Example – Encryption Program

TITLE Encryption Program

INCLUDE Irvine32.inc

KEY = 239 ; Any value Between 1-255

BUFMAX = 128; Maximum buffer size

.data

sPrompt BYTE "Enter the plain text:
", 0

sEncrypt BYTE "Cypher text: ", 0

sDecrypt BYTE "Decrypted: ", 0

buffer BYTE BUFMAX dup(0)

bufSize DWORD ?

```
.code
main PROC
    call    InputTheString
                ; input the plain text
    call    TranslateBuffer
                ; encrypt the buffer
    mov edx, OFFSET sEncrypt
                ; display encrypted message
    call    DisplayMessage
    call    TranslateBuffer
                ; decrypt the buffer
    mov edx, OFFSET sDecrypt
                ; display decrypted message
    call    DisplayMessage
    exit
main ENDP
```

```

;-----
InputTheString      PROC
;
; Asks the user to enter a string from the
; keyboard.  Saves the string and its length
; in variables
; Receives:  nothing
; Returns:   nothing
;-----

    pushad
    mov edx, OFFSET sPrompt ; display prompt
    call WriteString
    mov ecx, BUFMAX          ; maximum character count
    mov edx, OFFSET buffer   ; point to the buffer
    call ReadString          ; input the string
    mov bufsize, eax         ; save the length
    call CrLf
    popad
    ret
InputTheString      ENDP

```

```

;-----
DisplayMessage      PROC
;
; Displays the encrypted or decrypted
; message
; in variables
; Receives:  EDX points to the message
; Returns:   nothing
;-----
    pushad
    callWriteString
    mov edx, OFFSET buffer    ; display the buffer
    callWriteString
    callCrLF
    callCrLf
    popad
    ret
DisplayMessage      ENDP

```

```

;-----
TranslateBuffer      PROC
;
; Translate the string by exclusive-ORing
; each byte with the same integer
; Receives:  nothing
; Returns:   nothing
;-----

    pushad
    mov ecx, bufSize ; loop counter
    mov esi, 0       ; index 0 in buffer
L1:
    xor buffer[esi], KEY; translate a byte
    inc esi           ; point to next byte
    loop L1

    popad
    ret
TranslateBuffer      ENDP
END main

```

LOOPZ and LOOPE Instructions

- Syntax:
 LOOPE *destination*
 LOOPZ *destination*
- Logic:
 - $ECX \leftarrow ECX - 1$
 - if $ECX \neq 0$ and $ZF=1$, jump to *destination*
- The destination label must be between -128 and +127 bytes from the location of the following instruction
- Useful when scanning an array for the first element that meets some condition.

LOOPNZ and LOOPNE

- Syntax:

LOOPNZ *destination*

LOOPNE *destination*

- Logic:

- $ECX \leftarrow ECX - 1$;
- if $ECX \neq 0$ and $ZF=0$, jump to *destination*

LOOPZ and LOOPE Instructions :

Example

- Example

```
.data
intarray WORD 1, 20, 35, 012, 66, 40, 0
ArraySize=($-intarray)/2
.code
    mov     ebx, offset intarray ; point to the array
    sub     ebx, 2                ; back up one position
    mov     ecx, ArraySize        ; repeat 100 times
next:
    add     ebx, 2                ; point to next entry
    cmp     word ptr [ebx], 0     ; compare value to zero
    loopz   next                 ; loop while ZF 1, CX > 0
```

LOOPNZ example

The following code finds the first positive value in an array:

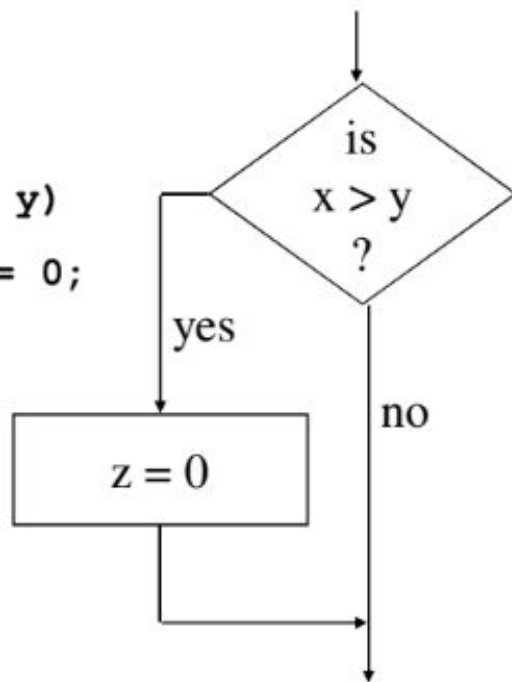
```
.data
array SWORD -3,-6,-1,-10,10,30,40,4
sentinel SWORD 0
.code
    mov esi,OFFSET array
    mov ecx,LENGTHOF array
next:
    test WORD PTR [esi],8000h    ; test sign bit
    pushfd                      ; push flags on stack
    add esi,TYPE array
    popfd                       ; pop flags from stack
    loopnz next                 ; continue loop
    jnz quit                   ; none found
    sub esi,TYPE array          ; ESI points to value
quit:
```

Writing IF-THEN

In C++:

```
if (x > y)
```

```
    z = 0;
```



In Assembler

```
mov ax, x
```

```
cmp ax, y
```

```
jng L1
```

```
mov ax, 0
```

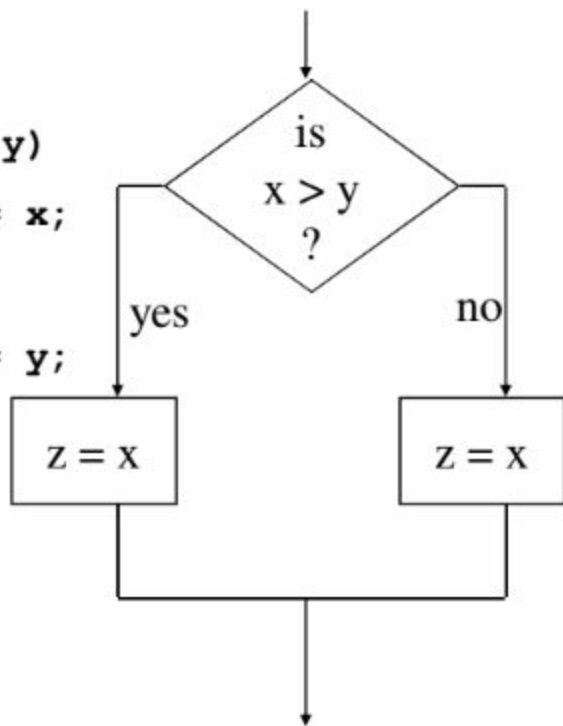
```
mov z, ax
```

```
L1:
```

Writing IF-THEN-ELSE

In C++:

```
if (x > y)
    z = x;
else
    z = y;
```



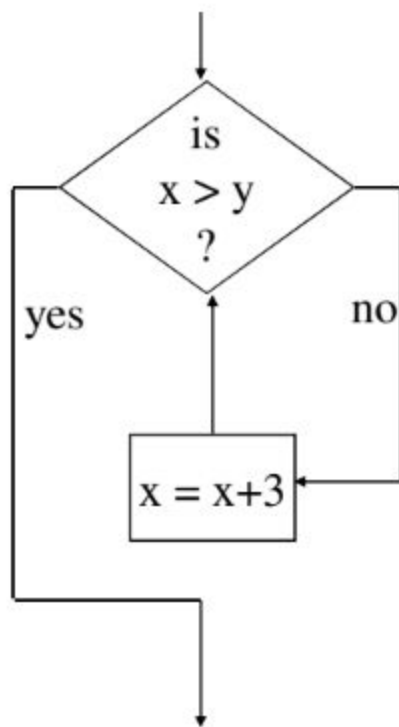
In Assembler

```
mov ax, x
cmp ax, y
jng L1
mov ax, x
jmp L2
L1:
mov ax, y
L2:
mov z, ax
```

Writing WHILE loops

In C++:

```
while (x <= y)  
    x = x + 3;
```



In Assembler

L1:

```
mov ax, x  
cmp ax, y  
jg L2  
mov ax, x  
add x, 3  
jmp L1
```

L2:

Conditional Structures

- An IF structure implies that a boolean expression is followed by two lists of statements;
- one performed when the expression is true, and another performed when the expression is false:

if(*boolean-expression*)

statement-list-1

else

statement-list-2

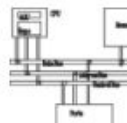
Block-structured IF statements



Assembly language programmers can easily translate logical statements written in C++/Java into assembly language. For example:

```
if( op1 == op2 )  
    x = 1;  
else  
    x = 2;
```

```
mov  eax,op1  
cmp  eax,op2  
jne  L1  
mov  X,1  
jmp  L2  
L1:  mov  X,2  
L2:
```



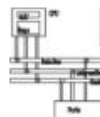
Example

Implement the following pseudocode in assembly language. All values are unsigned:

```
if( ebx <= ecx )
{
    eax = 5;
    edx = 6;
}
```

```
    cmp ebx,ecx
    ja  next
    mov eax,5
    mov edx,6
next:
```

Example

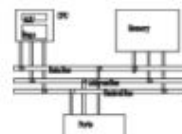


Implement the following pseudocode in assembly language. All values are 32-bit signed integers:

```
if( var1 <= var2 )
    var3 = 10;
else
{
    var3 = 6;
    var4 = 7;
}
```

```
mov eax,var1
cmp eax,var2
jle L1
mov var3,6
mov var4,7
jmp L2
L1: mov var3,10
L2:
```

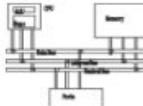
Compound expression with AND



- When implementing the logical AND operator, consider that HLLs use short-circuit evaluation
- In the following example, if the first expression is false, the second expression is skipped:

```
if (a1 > b1) AND (b1 > c1)  
    x = 1;
```

Compound expression with AND



```
if (a1 > b1) AND (b1 > c1)
    X = 1;
```

This is one possible implementation . . .

```
    cmp al,b1                ; first expression...
    ja  L1
    jmp next
L1:
    cmp bl,cl                ; second expression...
    ja  L2
    jmp next
L2:                          ; both are true
    mov X,1                  ; set X to 1
next:
```

Compound expression with AND

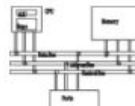


```
if (a1 > b1) AND (b1 > c1)
    X = 1;
```

But the following implementation uses 29% less code by reversing the first relational operator. We allow the program to "fall through" to the second expression:

```
    cmp al,b1          ; first expression...
    jbe next          ; quit if false
    cmp bl,cl          ; second expression...
    jbe next          ; quit if false
    mov X,1            ; both are true
next:
```

Exercise . . .



Implement the following pseudocode in assembly language. All values are unsigned:

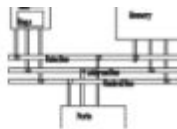
```
if( ebx <= ecx
  && ecx > edx )
{
    eax = 5;
    edx = 6;
}
```

```
cmp ebx,ecx
ja  next
cmp ecx,edx
jbe next
mov eax,5
mov edx,6

next:
```

(There are multiple correct solutions to this problem.)

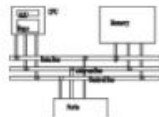
Compound Expression with OR



- In the following example, if the first expression is true, the second expression is skipped:

```
if (a1 > b1) OR (b1 > c1)
    x = 1;
```

Compound Expression with OR



```
if (a1 > b1) OR (b1 > c1)
    X = 1;
```

We can use "fall-through" logic to keep the code as short as possible:

```
    cmp al,b1      ; is AL > BL?
    ja  L1         ; yes
    cmp bl,c1      ; no: is BL > CL?
    jbe next       ; no: skip next statement
L1: mov X,1        ; set X to 1
next:
```

Example 2

- The following pseudocode statement has two branches:
if op1 > op2 then
call Routine1
else
call Routine2
end if
- In the following assembly language translation of the pseudocode, we assume that *op1* and *op2* are signed doubleword variables. When comparing variables, one must be moved to a register:

```
mov eax,op1 ; move op1 to a register  
cmp eax,op2 ; op1 > op2?  
jg A1 ; yes: call Routine1  
call Routine2 ; no: call Routine2  
jmp A2 ; exit the IF statement  
A1: call Routine1  
A2:
```